

(HATI)

(CampusFind)

Software Design Document

Name (s):

MUHAMMAD
TAUFIQ BIN
MOHD ARIFIN
IZZHILMY BIN
SHAMSUL
BAHRIN
NURHANIM
NABILA
BINTI AB
RAZAK
SUHAIL
AMANI
BINTI
MOHD
IKBAL

Section: S3G1

Date: 12/06/2026

Software Design Document

TABLE OF CONTENTS

Software Design Document.....	1
1.0 INTRODUCTION.....	3
1.1 Purpose.....	3
1.2 Scope.....	3
1.3 Overview.....	4
1.4 Reference Material.....	4
1.5 Definitions and Acronyms.....	5
2.0 SYSTEM OVERVIEW.....	9
3.0 SYSTEM ARCHITECTURE.....	10
3.1 Architectural Design.....	10
3.2 Decomposition Description.....	12
3.2.1 Authentication & Session Module.....	12
3.2.2 Report Management Module.....	12
3.2.3 Matching Engine.....	13
3.2.4 Claim & Verification Module.....	13
3.2.5 Notification Service.....	14
3.2.6 Admin Dashboard Module.....	14
3.3 Design Rationale.....	15
4.0 DATA DESIGN.....	17
4.1 Data Description.....	17
4.2 Data Dictionary.....	19
5.0 COMPONENT DESIGN.....	23
6.0 HUMAN INTERFACE DESIGN.....	30
6.1 Overview of User Interface.....	30
6.2 Screen Images.....	31
6.3 Screen Objects and Actions.....	40
7.0 REQUIREMENTS MATRIX.....	42
8.0 APPENDICES.....	52

1.0 INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) describes the architecture, system design, and data design of CampusFind, a web-based Campus Lost and Found Management System developed for Universiti Teknikal Malaysia Melaka (UTeM). The document translates the functional and non-functional requirements defined in the Software Requirements Specification (SRS) into concrete architectural decisions, module decompositions, and data structures that will guide the development team during implementation.

The intended audience for this document includes:

- Software Developers and Engineers as the primary guide for implementation of all modules and interfaces.
- University IT Department for understanding integration points with existing UTeM infrastructure (SSO, SMTP, database servers) and for future maintenance.
- Faculty Administrators and Campus Security Personnel to understand how the system is structured at a high level and how responsibilities are partitioned across subsystems.
- Project Supervisors and Evaluators for review of design decisions and their alignment with stated requirements.

1.2 Scope

CampusFind is a centralized, web-based lost and found management platform designed to replace the fragmented and informal item-recovery methods currently in use at UTeM (WhatsApp group posts, physical bulletin boards, manual security counter logs). The system serves registered university members students, staff, and faculty administrators who are authenticated via the university's existing Single Sign-On (SSO) infrastructure.

The key goals and objectives of the system are:

- Centralization: Provide a single, searchable digital platform for all lost and found item reports across the campus.
- Automation: Implement an intelligent matching engine that automatically cross-references lost and found reports and alerts relevant users when a potential match is detected.
- Verification: Enforce a structured ownership claim and verification process to ensure items are returned to their rightful owners.
- Accountability: Offer a Faculty Admin dashboard with full oversight, status management, and statistical reporting capabilities.
- Transparency: Allow users to track the lifecycle of their submitted reports through clearly defined statuses: MISSING → LOCATED → VERIFIED → RETURNED.

The system is scoped to the UTeM campus environment and physical security counters serve as the designated item custody and handover points. Guest or visitor access is outside

Software Design Document

the current scope.

1.3 Overview

This document is organized into four main sections:

Section 1 – Introduction covers the purpose, scope, references, and terminology of this document.

Section 2 – System Overview provides a narrative description of what CampusFind does, who it serves, and the broader context in which it operates.

Section 3 – System Architecture presents the three-tier architectural design, a decomposition of each subsystem, and the rationale behind key design decisions.

Section 4 – Data Design describes the data entities, their relationships, storage strategy, and a complete data dictionary.

1.4 Reference Material

The following documents and sources were used as references in the preparation of this SDD:

CampusFind Software Requirements Specification (SRS), Version 1.0 FTMK, UTeM, 2026. The primary source document from which all design decisions in this SDD are derived.

IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications. IEEE Computer Society, 1998.

Personal Data Protection Act (PDPA) 2010 (Act 709) Department of Personal Data Protection Malaysia (JPDP). Referenced for data privacy, user authentication, and protection of personal information.

Oracle Database Security Guide Oracle Corporation, 2023. Referenced for secure database management and access control practices.

Nielsen Norman Group 10 Usability Heuristics for User Interface Design Nielsen, J., 2020. Referenced to support accessible and intuitive UI design.

Google Firebase Cloud Messaging Documentation Google Developers, 2024. Referenced for real-time push notification architecture.

Kumar, R., & Sharma, P. (2021) Smart Lost and Found System Using Web Technology. ResearchGate. Referenced for automated item matching concepts..

1.5 Definitions and Acronyms

Attribute	Description
CampusFind	The Campus Lost and Found Management System described in this document.
UTeM	Universiti Teknikal Malaysia Melaka the institution for which this system is built.
SRS	Software Requirements Specification the requirements document from which this SDD is derived.
SDD	Software Design Document this document.
SSO	Single Sign-On a university-wide authentication service that allows users to log in with a single set of credentials.
LDAP	Lightweight Directory Access Protocol a protocol used to access and maintain distributed directory information; may underpin the UTeM Identity System.
SAML 2.0	Security Assertion Markup Language 2.0 an open standard for exchanging authentication and authorization data between identity providers and service providers.
RBAC	Role-Based Access Control a security model that restricts system access based on the roles of individual users.

Software Design Document

MVC	Model-View-Controller a software architectural pattern that separates application logic into three components: Model (data), View (UI), and Controller (logic).
REST	Representational State Transfer an architectural style for designing networked applications via HTTP.
API	Application Programming Interface a set of definitions and protocols allowing software components to communicate.
ORM	Object-Relational Mapping a technique for converting data between incompatible systems (object-oriented code and relational databases) using a programming language.
DBMS	Database Management System the software used to manage database storage and retrieval (e.g., MySQL, PostgreSQL).
HTTPS/TLS	HyperText Transfer Protocol Secure / Transport Layer Security protocols ensuring encrypted communication between client and server.
SMTP	Simple Mail Transfer Protocol a standard protocol for sending email.
FCM	Firebase Cloud Messaging Google's cross-platform messaging solution for push notifications.
PDPA	Personal Data Protection Act 2010 (Malaysia) Malaysian law governing the collection, processing, and storage of personal data.

Software Design Document

WCAG 2.1	Web Content Accessibility Guidelines 2.1 internationally recognized guidelines for making web content accessible to people with disabilities.
Faculty Admin	A user with administrative privileges over the CampusFind system, typically a designated staff member within a faculty.
iLost Report	A report submitted by an authenticated user describing an item they have lost.
Found Report	A report submitted by an authenticated user describing an item they have discovered on campus.
Claimant	An authenticated user who submits a claim asserting ownership of a found item.
Finder	An authenticated user who submitted the found item report.
Matching Engine	The subsystem responsible for automatically cross-referencing lost and found reports to identify potential matches.
In-App Notification	A notification message displayed within the CampusFind web interface.
MISSING	Item status: a lost report has been submitted but no matching found item has been identified.

Software Design Document

LOCATED	Item status: a potential match between a lost and found report has been identified by the system.
VERIFIED	Item status: the claimant's ownership has been confirmed through the verification process.
RETURNED	Item status: the physical item has been handed over to the rightful owner and all associated reports are closed.
DXA	Document Extended Attribute units (used in document formatting; included here for completeness, not system-relevant).

2.0 SYSTEM OVERVIEW

CampusFind addresses a clearly identified operational gap at UTeM: there is currently no centralized, reliable system for managing lost and found items on campus. Students, staff, and lecturers frequently lose personal belongings across lecture halls, cafeterias, libraries, and open spaces. The existing informal channels WhatsApp group posts, physical bulletin boards, and manual visits to the security counter are ephemeral, fragmented, and ineffective. Items go unclaimed because owners are unaware they have been found, and security personnel have no consistent logging method.

CampusFind is a web-based platform that formalizes and digitizes this process end-to-end. The system integrates with UTeM's existing SSO infrastructure to authenticate users without managing its own credentials, ensuring that only verified university members can access the platform. Upon login, users are assigned one of three roles: Student/Staff, Campus Security, or Faculty Admin each with a tailored interface and permission set.

The core workflow is as follows: when a user loses an item, they submit a Lost Report describing the item's category, location last seen, date, and an optional photo. When a user finds an item, they submit a Found Report with similar details and a mandatory photo. Upon each submission, the system's Matching Engine automatically cross-references the new report against existing reports of the opposite type, comparing category, location, and date proximity. If a potential match is found, both the owner and the finder are notified via email and in-app notification, with a confidence score indicating the likelihood of a match.

The finder reviews any claim submitted by a potential owner. The claimant must answer identifying verification questions to prove ownership. If the finder approves the claim, both parties are directed to the security counter for physical handover. The Campus Security team confirms the handover, and the system automatically updates all associated report statuses to RETURNED and removes them from active search listings.

Faculty Admins have oversight of all reports on the platform. They can update statuses, remove invalid or duplicate reports, notify affected users, and generate statistical summary reports covering total lost, found, and returned items as well as high-loss campus locations.

Non-functionally, the system is designed to handle up to 300 concurrent users and 1,000 active reports, returning search results within one second. All data transmission is encrypted via HTTPS/TLS, sessions expire after 15 minutes of inactivity, and all administrative actions are logged for audit compliance with PDPA 2010.

3.0 SYSTEM ARCHITECTURE

3.1 Architectural Design

CampusFind is designed using a Three-Tier Architecture, separating the system into a Presentation Tier, an Application (Business Logic) Tier, and a Data Tier. This separation ensures that each layer can be developed, tested, and maintained independently, supports scalability, and aligns with the modular maintainability requirement stated in the SRS.

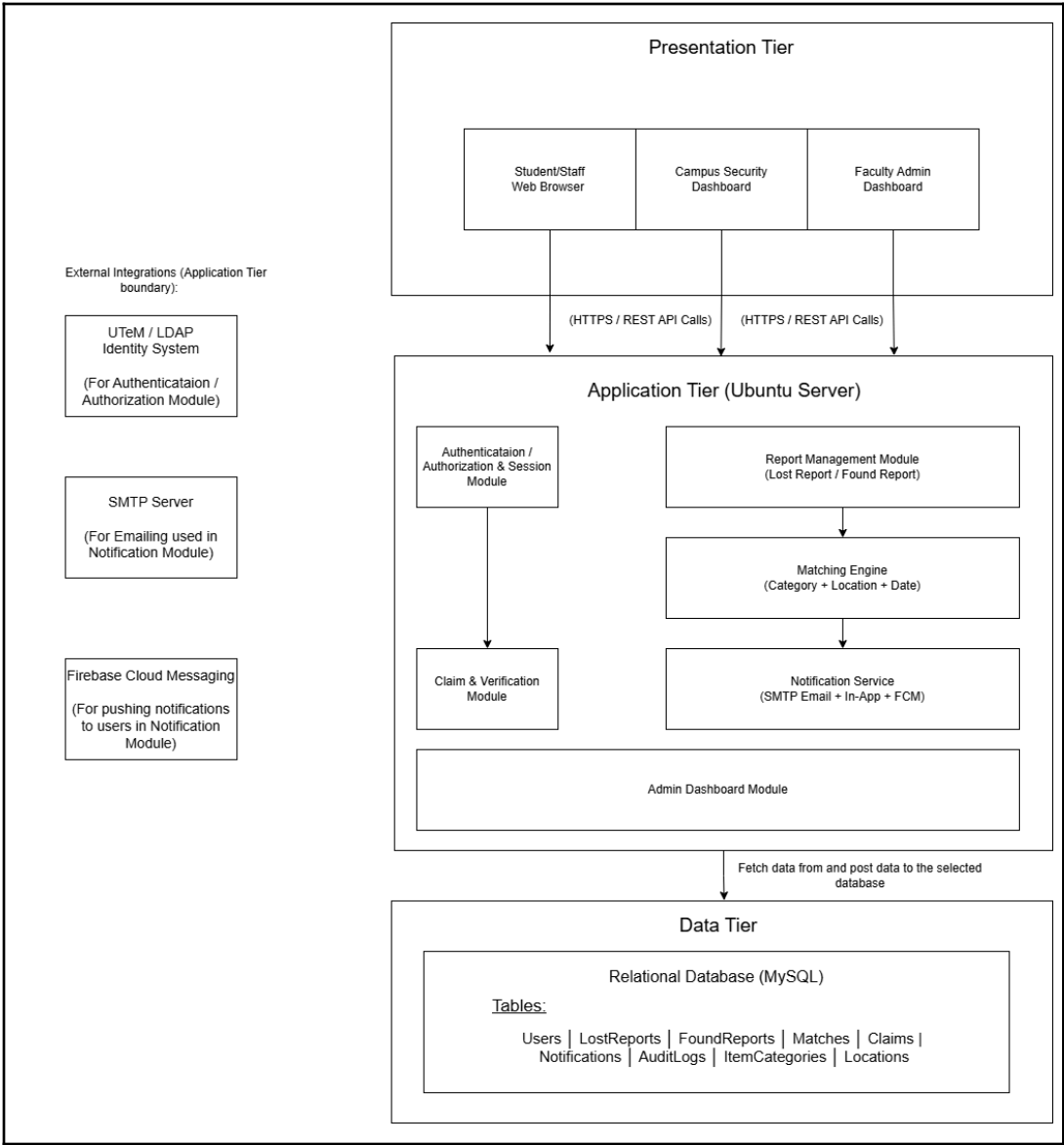
The system is further decomposed within the Application Tier into six discrete subsystems, each responsible for a distinct area of functionality. These subsystems communicate with one another through well-defined internal interfaces and collectively serve the Presentation Tier through a RESTful API layer.

Subsystems and Responsibilities:

Subsystem	Primary Responsibility
Authentication & Session Module	Validates user credentials via SSO (LDAP/SAML 2.0), assigns user roles, manages session tokens and expiry.
Report Management Module	Handles creation, editing, retrieval, and closure of Lost and Found item reports.
Matching Engine	Automatically cross-references lost and found reports by category, location, and date proximity; calculates confidence scores.
Claim & Verification Module	Manages the submission, review, approval, and rejection of ownership claims; tracks handover status.
Notification Service	Sends email (via SMTP) and in-app notifications for match alerts, claim events, and admin actions.
Admin Dashboard Module	Provides Faculty Admin with full report oversight, manual status management, report removal, and statistical summary generation.

Software Design Document

Architecture Diagram:



Subsystem Interactions:

When a user submits a Found Report, the Report Management Module saves the report to the database and immediately invokes the Matching Engine.

The Matching Engine queries the database for compatible Lost Reports and, upon identifying a match, records the match and triggers the Notification Service. The Notification Service dispatches an email via the SMTP server and records an in-app notification.

If the matched owner submits a claim, the Claim & Verification Module manages the claim lifecycle and again triggers the Notification Service for relevant alerts.

Software Design Document

The Admin Dashboard Module has read/write access to all report and match records and interfaces with the Notification Service when performing administrative removals.

3.2 Decomposition Description

This section provides an object-oriented decomposition of each subsystem, describing the key objects, their attributes and responsibilities, and the relationships between them.

3.2.1 Authentication & Session Module

Responsibility: Authenticate users via the UTeM Identity System, retrieve role information, create and manage session tokens, and enforce session expiry.

Key Objects:

- **AuthController** receives login requests from the Presentation Tier, forwards credentials to the external UTeM Identity System via LDAP/SAML 2.0, processes the response, and establishes a session.
- **SessionManager** Generates, stores, and validates session tokens. Enforces automatic expiry after 15 minutes of inactivity.
- **UserProfile** A data transfer object populated from the UTeM Identity System response, containing the user's ID, full name, faculty, and role (Student, Staff, or Faculty Admin).

Key Interactions:

On a login request, AuthController calls the external UTeM SSO endpoint. A validated response populates a UserProfile object, which SessionManager binds to a newly created session token. The token is returned to the Presentation Tier and included in all subsequent API calls for authorization checks.

3.2.2 Report Management Module

Responsibility: Handle the full lifecycle of Lost and Found item reports, including creation, validation, retrieval, editing, and closure.

Key Objects:

- **ReportController** Exposes REST API endpoints for creating, reading, updating, and deleting (CRUD) reports. Enforces that only the report owner or a Faculty Admin may edit or close a given report.
- **LostReport** Represents a lost item report entity with fields for title, category, description, last-known location, date lost, optional time, optional photo URL, status, and owner reference.
- **FoundReport** Represents a found item report entity with fields for title, category, description, found location, date found, optional time, mandatory photo URL, current storage location, status, and finder reference.
- **ReportValidator** Validates incoming report data before persistence, enforcing mandatory fields (e.g., photo required for Found Reports) and acceptable file

Software Design Document

types/sizes.

- ImageStorageService Handles upload, validation (type and size $\leq 2\text{MB}$), and storage of item photos on the server file system or cloud storage bucket.

Key Interactions:

On report submission, ReportController passes data to ReportValidator. If validation passes, the report is persisted and ReportController calls the Matching Engine asynchronously. If the image upload fails, ImageStorageService returns an error that ReportController surfaces to the client.

3.2.3 Matching Engine

Responsibility: Automatically cross-reference newly submitted reports against existing reports of the opposite type to identify potential matches; calculate and record a confidence score for each match.

Key Objects:

- MatchingEngine The core service invoked asynchronously by the Report Management Module after any new report is persisted.
- MatchCriteria Encapsulates the matching logic: category must match exactly; location must be the same or adjacent; date difference must be within ± 7 days.
- ConfidenceCalculator Assigns a confidence score (e.g., High / Medium / Low) to each potential match based on how many criteria are satisfied and the specificity of the matching description keywords.
- Match Represents an identified pairing between one LostReport and one FoundReport, storing the confidence score and current match status.

Key Interactions:

After a new report is saved, MatchingEngine queries the database for candidate reports of the opposite type that satisfies MatchCriteria. For each candidate, ConfidenceCalculator computes a score. All qualifying matches are persisted as Match records and the Notification Service is called for each.

3.2.4 Claim & Verification Module

Responsibility: Manage the submission of ownership claims, enforce completion of verification questions, facilitate finder review and approve/reject decisions, and update report statuses through to RETURNED.

Key Objects:

- ClaimController Exposes API endpoints for submitting, retrieving, approving, and rejecting claims.
- Claim Represents an ownership claim entity containing a reference to the FoundReport, the claimant user, verification question answers, and claim status (Pending Review / Approved / Rejected).
- VerificationForm Defines the set of identifying questions the claimant must answer (e.g., brand, distinguishing marks, contents). Enforces that all fields are completed

Software Design Document

before submission.

- **HandoverManager** Updates report statuses when the finder marks an item as "Handed Over," triggering automatic closure of both the associated LostReport and FoundReport and updating their statuses to RETURNED.

Key Interactions:

On claim submission, ClaimController validates through VerificationForm and persists the Claim, then calls the Notification Service to alert the finder. When the finder approves, ClaimController updates the claim status and calls the Notification Service for both parties, then provides handover location details. When the item is marked as handed over, HandoverManager closes both reports and records the event.

3.2.5 Notification Service

Responsibility: Dispatch email notifications via SMTP and in-app notifications for all system events (match found, claim submitted, claim approved/rejected, report removed by admin, admin reminders).

Key Objects:

- **NotificationService** The central service called by all other subsystems when a notification event occurs.
- **EmailNotifier** Formats and sends emails via the configured SMTP server (e.g., SendGrid). Implements a retry mechanism on failure.
- **InAppNotifier** Persists notification records to the database for retrieval and display within the CampusFind interface.
- **PushNotifier (optional)** Integrates with Firebase Cloud Messaging to send browser push notifications. Degrades gracefully to email if FCM is unavailable.
- **NotificationRecord** A persisted entity representing an in-app notification, with fields for recipient, message body, event type, timestamp, and read status.

Key Interactions:

Any subsystem (Matching Engine, Claim Module, Admin Dashboard Module) calls NotificationService with an event type and recipient list. NotificationService delegates to EmailNotifier, InAppNotifier, and optionally PushNotifier in parallel. In-app notifications are stored as NotificationRecord entries and surfaced on the user's dashboard.

3.2.6 Admin Dashboard Module

Responsibility: Provide Faculty Admin users with a centralized management interface for viewing all reports, manually updating statuses, removing invalid reports, and generating summary analytics.

Key Objects:

- **AdminController** Exposes Admin-only API endpoints; enforces that all calls are made by a user with the Faculty Admin role.

Software Design Document

- **ReportManager** Allows the Admin to update the status of any report or remove it from the system. Calls NotificationService when a report is removed.
- **AnalyticsService** Queries the database to compile summary statistics for a specified date range: total lost reports, total found reports, total returned items, and high-loss campus locations.
- **SummaryReport** A data transfer object encapsulating compiled analytics, available for in-browser display and download (e.g., CSV or PDF export).
- **AuditLogger** Records all significant administrative actions (status updates, removals, claim interventions) with timestamps and actor identity for audit and security monitoring.

3.3 Design Rationale

Three-Tier Architecture

The three-tier architecture was selected because it cleanly separates user interface concerns from business logic and data storage. This separation is essential for CampusFind given its multiple user roles (students/staff, security, admin) each requiring a different interface, while sharing the same underlying data and business rules. It also allows the Application Tier to be scaled independently of the Presentation Tier during peak university hours, addressing the SRS performance requirement of 300 concurrent users.

An alternative considered was a two-tier (client-server) architecture where the client directly accesses the database. This was rejected because it would expose database credentials to the client, directly violating the PDPA 2010 compliance and RBAC security requirements of the SRS.

Relational Database (MySQL)

A relational DBMS was chosen because the CampusFind data is inherently structured with strong referential relationships between entities (a Claim must reference a FoundReport, a Match must reference both a LostReport and a FoundReport, Notifications must reference Users). A relational model enforces these relationships through foreign key constraints, preventing data integrity issues such as orphaned claims. The SRS explicitly specifies MySQL 8.0 and above.

A NoSQL document store (e.g., MongoDB) was considered for its schema flexibility and ease of horizontal scaling. However, it was rejected because the well-defined and stable data relationships in CampusFind do not benefit from schema flexibility, and the team's maintainability requirement favors a DBMS familiar to the UTeM IT department.

SSO-Only Authentication (No Local Credentials)

The decision not to implement a local user registration or credential storage system was made in direct response to the PDPA 2010 constraint and the university's need for verified identity. By delegating authentication entirely to the UTeM Identity System via LDAP/SAML 2.0, CampusFind never stores user passwords and inherits the university's existing identity assurance. This also ensures that deactivated university accounts automatically lose access without any action required by CampusFind administrators.

Asynchronous Matching Engine

Software Design Document

The Matching Engine is invoked asynchronously immediately after a report is saved, rather than blocking the report submission response. This ensures users receive instant confirmation of their submission (within the SRS one-second response target) while matching which may involve querying across up to 1,000 active reports completes independently. The match notification is then delivered within the five-second SRS target.

Modular Subsystem Design

Each functional area is encapsulated in its own module with a defined responsibility boundary. This was chosen to satisfy the SRS maintainability quality attribute: future enhancements (such as a guest inquiry portal, mobile app, or AI-based description matching) can be added as new modules or as extensions to existing ones without requiring significant changes to other parts of the system.

4.0 DATA DESIGN

4.1 Data Description

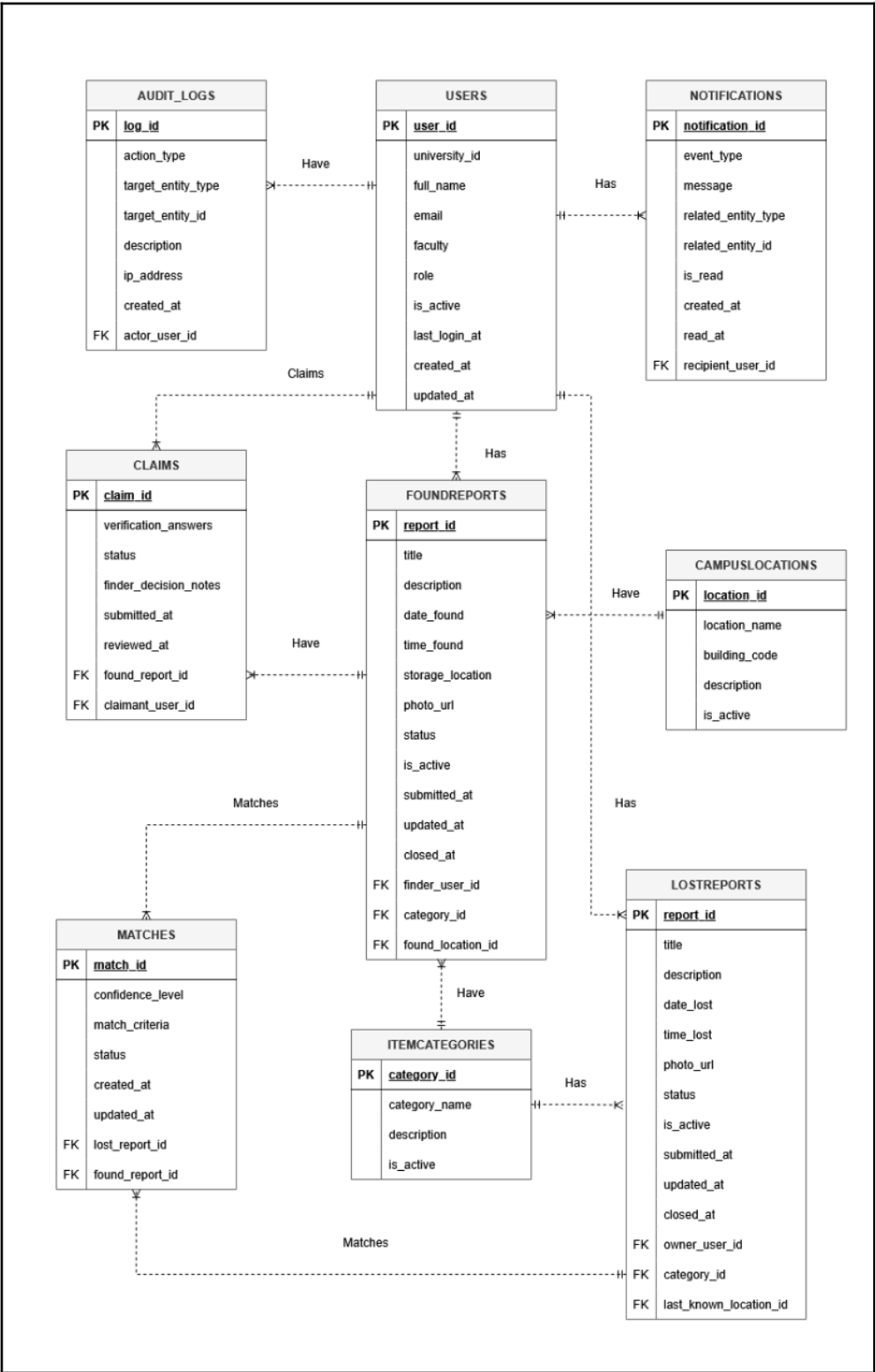
CampusFind uses a centralized relational database (MySQL 8.0+ or PostgreSQL 14+) as its sole persistent data store. All system entities user profiles, reports, matches, claims, notifications, and audit logs are stored as structured relational tables.

Core Entities and Their Purpose:

- **Users** Stores profile information retrieved from the UTeM Identity System upon first authentication. Users are not created by CampusFind directly; the record is created or updated on SSO login. Each user has a role (Student, Staff, or Faculty Admin) that governs their access.
- **LostReports** Stores all lost item reports submitted by authenticated users. Each report is associated with an owner (User), assigned a status, and linked to any matches identified by the Matching Engine.
- **FoundReports** Stores all found item reports submitted by authenticated users. Each report requires a mandatory photo and is assigned a status. Found reports may be linked to one or more Claims.
- **ItemCategories** A reference table of item categories (e.g., Electronics, Stationery, Clothing, Accessories, Wallet/Cash, ID/Card, Key, Bag, Other) used to standardize categorization across reports and enable category-based matching and filtering.
- **CampusLocations** A reference table of named campus locations within UTeM (e.g., Faculty buildings, cafeterias, libraries, open areas) used to standardize location data across reports and support proximity-based matching and high-loss location analytics.
- **Matches** Records identified pairings between a LostReport and a FoundReport, including the confidence score and current match status. A LostReport may have multiple candidate Matches.
- **Claims** Records ownership claim submissions by authenticated users against a specific FoundReport. Each Claim stores the verification answers provided by the claimant and the finder's decision (Pending, Approved, Rejected).
- **Notifications** Stores in-app notification records for each user, including the event type, message, associated entity references, read status, and timestamp. Email delivery is handled at the application layer and not persisted in detail; only a delivery attempt log is stored.
- **AuditLogs** Records all significant system events for security monitoring and PDPA compliance, particularly administrative actions such as status overrides, report removals, and claim interventions.

Software Design Document

Entity Relationship Summary:



Software Design Document

A User can author many LostReports and many FoundReports.

A LostReport belongs to one User (owner) and can be involved in many Matches.

A FoundReport belongs to one User (finder), can be involved in many Matches, and can receive many Claims.

A Match links exactly one LostReport and one FoundReport.

A Claim links one FoundReport to one User (claimant).

Notifications belong to one User. AuditLogs belong to one User (the actor who performed the action).

Storage and Processing Approach:

All timestamps are stored in UTC and displayed in Malaysia Standard Time (UTC+8) at the Presentation Tier. Image files are stored on the server file system or a cloud object storage bucket, with only the file path/URL stored in the database. All personally identifiable information (matric numbers, staff IDs, email addresses) stored in the Users table is encrypted at rest in compliance with PDPA 2010. The database is backed up daily with automated jobs, and all tables use primary keys and foreign key constraints to maintain referential integrity.

4.2 Data Dictionary

AuditLogs

Attribute	Type	Description
log_id	INT (PK, AUTO_INCREMENT)	Unique identifier for each audit log entry.
actor_user_id	INT (FK → Users.user_id)	User who performed the action.
action_type	VARCHAR(100)	Type of action performed.
target_entity_type	VARCHAR(50)	Type of entity affected.
target_entity_id	INT	ID of the affected entity.
description	TEXT	Human-readable description.
ip_address	VARCHAR(45)	IP address of action source.
created_at	DATETIME	Timestamp when recorded.

CampusLocations

Attribute	Type	Description
location_id	INT (PK, AUTO_INCREMENT)	Unique identifier.
location_name	VARCHAR(150)	Location display name.
building_code	VARCHAR(20)	Building code.

Software Design Document

description	TEXT	Optional notes.
is_active	BOOLEAN	Whether location is active.

Claims

Attribute	Type	Description
claim_id	INT (PK, AUTO_INCREMENT)	Unique identifier.
found_report_id	INT (FK)	Found report being claimed.
claimant_user_id	INT (FK)	Claimant user.
verification_answers	TEXT (JSON)	Verification answers.
status	ENUM	Claim status.
finder_decision_notes	TEXT	Finder notes.
submitted_at	DATETIME	Submission timestamp.
reviewed_at	DATETIME	Review timestamp.

FoundReports

Attribute	Type	Description
report_id	INT (PK)	Unique report ID.
finder_user_id	INT (FK)	Finder user.
title	VARCHAR(200)	Item title.
category_id	INT (FK)	Item category.
description	TEXT	Item description.
found_location_id	INT (FK)	Found location.
date_found	DATE	Date found.
time_found	TIME	Time found.
storage_location	VARCHAR(255)	Current storage location.
photo_url	VARCHAR(500)	Photo URL/path.
status	ENUM	Report status.
is_active	BOOLEAN	Visibility status.
submitted_at	DATETIME	Created timestamp.
updated_at	DATETIME	Updated timestamp.
closed_at	DATETIME	Closed timestamp.

ItemCategories

Attribute	Type	Description
category_id	INT (PK)	Unique category ID.
category_name	VARCHAR(100)	Category name.

Software Design Document

description	TEXT	Category description.
is_active	BOOLEAN	Active flag.

LostReports

Attribute	Type	Description
report_id	INT (PK)	Unique report ID.
owner_user_id	INT (FK)	Owner user.
title	VARCHAR(200)	Item title.
category_id	INT (FK)	Item category.
description	TEXT	Item description.
last_known_location_id	INT (FK)	Last known location.
date_lost	DATE	Date lost.
time_lost	TIME	Time lost.
photo_url	VARCHAR(500)	Optional photo.
status	ENUM	Report status.
is_active	BOOLEAN	Visibility status.
submitted_at	DATETIME	Created timestamp.
updated_at	DATETIME	Updated timestamp.
closed_at	DATETIME	Closed timestamp.

Matches

Attribute	Type	Description
match_id	INT (PK)	Unique match ID.
lost_report_id	INT (FK)	Lost report.
found_report_id	INT (FK)	Found report.
confidence_level	ENUM	Confidence level.
matched_criteria	TEXT (JSON)	Matched criteria.
status	ENUM	Match status.
created_at	DATETIME	Created timestamp.
updated_at	DATETIME	Updated timestamp.

Notifications

Attribute	Type	Description
notification_id	INT (PK)	Unique notification ID.
recipient_user_id	INT (FK)	Recipient user.
event_type	VARCHAR(100)	Trigger event.
message	TEXT	Notification message.
related_entity_type	VARCHAR(50)	Related entity type.

Software Design Document

related_entity_id	INT	Related entity ID.
is_read	BOOLEAN	Read status.
created_at	DATETIME	Created timestamp.
read_at	DATETIME	Read timestamp.

Users

Attribute	Type	Description
user_id	INT (PK)	Unique user ID.
university_id	VARCHAR(20)	Matric/staff ID.
full_name	VARCHAR(200)	Full name.
email	VARCHAR(255)	Email address.
faculty	VARCHAR(150)	Faculty/department.
role	ENUM	System role.
is_active	BOOLEAN	Account active status.
last_login_at	DATETIME	Last login.
created_at	DATETIME	Created timestamp.
updated_at	DATETIME	Updated timestamp.

5.0 COMPONENT DESIGN

This section provides procedural-level descriptions (pseudocode / PDL) for the key member functions of every object identified in Section 3.2. Local variables are noted where relevant.

5.1 Authentication & Session Module

5.1.1 AuthController.login(credentials)

Purpose: Validate user via SSO and establish a session token.

```
FUNCTION login(credentials):
  IF credentials.username IS EMPTY OR credentials.password IS EMPTY THEN
    RETURN error("Username and password are required")
  END IF
  ssoResponse ← UTeM_SSO.authenticate(credentials)
  IF ssoResponse.status == FAILURE THEN
    LOG failed login attempt (username, timestamp, ip_address)
    RETURN error("Invalid credentials")
  END IF
  userProfile ← UserProfile.fromSSO(ssoResponse)
  IF userProfile.is_active == FALSE THEN
    RETURN error("Account is deactivated")
  END IF
  sessionToken ← SessionManager.createSession(userProfile)
  UPDATE Users SET last_login_at = NOW() WHERE university_id = credentials.username
  RETURN success(sessionToken, userProfile.role)
```

5.1.2 SessionManager.createSession(userProfile)

Purpose: Generate a secure session token and store with expiry.

```
FUNCTION createSession(userProfile):
  token ← generateSecureRandomToken(256-bit)
  expiresAt ← NOW() + 15 minutes
  STORE session { token, userProfile.user_id, userProfile.role, expiresAt } in
  session store
  RETURN token
```

5.1.3 SessionManager.validateSession(token)

```
FUNCTION validateSession(token):
  session ← SESSION_STORE.get(token)
  IF session IS NULL THEN RETURN error("Session not found")
  IF NOW() > session.expiresAt THEN
    SESSION_STORE.delete(token)
    RETURN error("Session expired")
  END IF
  session.expiresAt ← NOW() + 15 minutes // sliding window refresh
  RETURN session.userProfile
```

5.2 Report Management Module

5.2.1 ReportController.createLostReport(data, userProfile)

```
FUNCTION createLostReport(data, userProfile):
  errors ← ReportValidator.validateLostReport(data)
  IF errors IS NOT EMPTY THEN RETURN error(errors)
  IF data.photo IS PROVIDED THEN
    photoURL ← ImageStorageService.upload(data.photo)
    IF photoURL IS NULL THEN RETURN error("Image upload failed")
  END IF
  report ← INSERT INTO LostReports(
    owner_user_id, title, category_id, description,
    last_known_location_id, date_lost, time_lost,
    photo_url, status='MISSING', submitted_at=NOW())
  ASYNC INVOKE MatchingEngine.runForReport(report.report_id, 'LOST')
  RETURN success(report.report_id)
```

5.2.2 ReportController.createFoundReport(data, userProfile)

```
FUNCTION createFoundReport(data, userProfile):
  errors ← ReportValidator.validateFoundReport(data)
  IF errors IS NOT EMPTY THEN RETURN error(errors)
  photoURL ← ImageStorageService.upload(data.photo) // mandatory
  IF photoURL IS NULL THEN RETURN error("Photo is required for found reports")
  report ← INSERT INTO FoundReports(
    finder_user_id, title, category_id, description,
    found_location_id, date_found, time_found, storage_location,
    photo_url, status='LOCATED', submitted_at=NOW())
  ASYNC INVOKE MatchingEngine.runForReport(report.report_id, 'FOUND')
  RETURN success(report.report_id)
```

5.2.3 ReportValidator.validateLostReport(data)

```
FUNCTION validateLostReport(data):
  errors ← []
  IF data.title IS EMPTY THEN errors.add("Title is required")
  IF data.category_id IS EMPTY THEN errors.add("Category is required")
  IF data.last_known_location_id IS EMPTY THEN errors.add("Location is required")
  IF data.date_lost IS EMPTY THEN errors.add("Date lost is required")
  IF data.description LENGTH < 10 THEN errors.add("Description too short")
  IF data.photo IS PROVIDED THEN
    IF data.photo.size > 2MB THEN errors.add("Photo exceeds 2MB limit")
    IF data.photo.type NOT IN ['image/jpeg', 'image/png', 'image/webp']
      THEN errors.add("Invalid photo format")
  END IF
  RETURN errors
```

5.2.4 ImageStorageService.upload(file)

```
FUNCTION upload(file):
  IF file.size > 2MB THEN RETURN NULL
  IF file.type NOT IN ALLOWED_TYPES THEN RETURN NULL
  uniqueName ← generateUUID() + getExtension(file.type)
  TRY
    SAVE file TO storage_path/uniqueName
    RETURN storage_base_url + uniqueName
  CATCH IOException
    LOG error
    RETURN NULL
```

5.3 Matching Engine

5.3.1 MatchingEngine.runForReport(reportId, reportType)

```
FUNCTION runForReport(reportId, reportType):
  IF reportType == 'FOUND' THEN
    newReport ← SELECT * FROM FoundReports WHERE report_id = reportId
    candidates ← SELECT * FROM LostReports
                  WHERE status = 'MISSING' AND is_active = TRUE
                  AND category_id = newReport.category_id
  ELSE // 'LOST'
    newReport ← SELECT * FROM LostReports WHERE report_id = reportId
    candidates ← SELECT * FROM FoundReports
                  WHERE status IN ('LOCATED', 'VERIFIED') AND is_active = TRUE
                  AND category_id = newReport.category_id
  END IF

  FOR EACH candidate IN candidates:
    IF MatchCriteria.isLocationMatch(newReport, candidate) AND
       MatchCriteria.isDateMatch(newReport, candidate) THEN
      score ← ConfidenceCalculator.calculate(newReport, candidate)
      existingMatch ← SELECT * FROM Matches
                      WHERE lost_report_id = ... AND found_report_id = ...
      IF existingMatch IS NULL THEN
        INSERT INTO Matches (lost_report_id, found_report_id,
                             confidence_level, matched_criteria,
                             status='PENDING', created_at=NOW())
        NotificationService.notify('MATCH_FOUND', [ownerUserId, finderUserId])
      END IF
    END IF
  END FOR
```

5.3.2 MatchCriteria.isLocationMatch(r1, r2)

```
FUNCTION isLocationMatch(r1, r2):
  loc1 ← getLocation(r1) // CampusLocations record
  loc2 ← getLocation(r2)
  IF loc1.location_id == loc2.location_id THEN RETURN TRUE
  IF loc1.building_code == loc2.building_code THEN RETURN TRUE // adjacent areas
  RETURN FALSE
```

5.3.3 MatchCriteria.isDateMatch(r1, r2)

```
FUNCTION isDateMatch(r1, r2):
  dateLost ← r1.date_lost OR r2.date_lost
  dateFound ← r1.date_found OR r2.date_found
  diff ← ABS(dateLost - dateFound) in days
  RETURN diff <= 7
```

5.3.4 ConfidenceCalculator.calculate(r1, r2)

```
FUNCTION calculate(r1, r2):
  score ← 0
  // Category already matched (pre-filtered)
  score ← score + 40

  // Exact location match
  IF getLocationId(r1) == getLocationId(r2) THEN score ← score + 30
  ELSE score ← score + 15 // building match

  // Date proximity
  diff ← ABS(r1.date - r2.date)
  IF diff <= 1 THEN score ← score + 20
```

Software Design Document

```
ELSE IF diff <= 3 THEN score ← score + 10

// Description keyword overlap
shared ← keywordIntersection(r1.description, r2.description)
IF shared.count >= 3 THEN score ← score + 10

IF score >= 80 THEN RETURN 'HIGH'
IF score >= 55 THEN RETURN 'MEDIUM'
RETURN 'LOW'
```

5.4 Claim & Verification Module

5.4.1 ClaimController.submitClaim(foundReportId, claimantId, answers)

```
FUNCTION submitClaim(foundReportId, claimantId, answers):
    foundReport ← SELECT * FROM FoundReports WHERE report_id = foundReportId
    IF foundReport IS NULL OR foundReport.is_active == FALSE THEN
        RETURN error("Found report not available")
    END IF
    IF foundReport.finder_user_id == claimantId THEN
        RETURN error("Cannot claim your own found report")
    END IF
    existingClaim ← SELECT * FROM Claims
                        WHERE found_report_id = foundReportId AND claimant_user_id =
claimantId
    IF existingClaim IS NOT NULL THEN
        RETURN error("You have already submitted a claim for this item")
    END IF
    errors ← VerificationForm.validate(answers)
    IF errors IS NOT EMPTY THEN RETURN error(errors)
    claim ← INSERT INTO Claims(
                found_report_id, claimant_user_id,
                verification_answers=JSON(answers),
                status='PENDING_REVIEW', submitted_at=NOW())
    NotificationService.notify('CLAIM_SUBMITTED', [foundReport.finder_user_id])
    RETURN success(claim.claim_id)
```

5.4.2 ClaimController.approveClaim(claimId, finderUserId, notes)

```
FUNCTION approveClaim(claimId, finderUserId, notes):
    claim ← SELECT * FROM Claims WHERE claim_id = claimId
    IF claim IS NULL THEN RETURN error("Claim not found")
    foundReport ← SELECT * FROM FoundReports WHERE report_id = claim.found_report_id
    IF foundReport.finder_user_id != finderUserId THEN
        RETURN error("Unauthorized")
    END IF
    UPDATE Claims SET status='APPROVED', finder_decision_notes=notes,
                        reviewed_at=NOW() WHERE claim_id = claimId
    // Reject all other pending claims for same found report
    UPDATE Claims SET status='REJECTED' WHERE found_report_id = claim.found_report_id
                        AND claim_id != claimId
    NotificationService.notify('CLAIM_APPROVED', [claim.claimant_user_id,
finderUserId])
    RETURN success(securityCounter: 'Please proceed to the nearest security counter')
```

5.4.3 HandoverManager.confirmHandover(claimId, securityStaffId)

```
FUNCTION confirmHandover(claimId, securityStaffId):
    claim ← SELECT * FROM Claims WHERE claim_id = claimId AND status = 'APPROVED'
    IF claim IS NULL THEN RETURN error("Approved claim not found")
    foundReport ← SELECT * FROM FoundReports WHERE report_id = claim.found_report_id
    lostReport ← SELECT lr.* FROM LostReports lr
                    JOIN Matches m ON m.lost_report_id = lr.report_id
```

Software Design Document

```
WHERE m.found_report_id = claim.found_report_id LIMIT 1
NOW_TS ← NOW()
UPDATE FoundReports SET status='RETURNED', is_active=FALSE, closed_at=NOW_TS
WHERE report_id = claim.found_report_id
IF lostReport IS NOT NULL THEN
  UPDATE LostReports SET status='RETURNED', is_active=FALSE, closed_at=NOW_TS
  WHERE report_id = lostReport.report_id
END IF
UPDATE Matches SET status='RESOLVED' WHERE found_report_id = claim.found_report_id
AuditLogger.log(securityStaffId, 'HANDOVER_CONFIRMED', 'CLAIM', claimId)
NotificationService.notify('ITEM_RETURNED', [claim.claimant_user_id,
foundReport.finder_user_id])
RETURN success()
```

5.5 Notification Service

5.5.1 NotificationService.notify(eventType, recipientIds, context)

```
FUNCTION notify(eventType, recipientIds, context):
  template ← getTemplate(eventType) // subject + message body
  FOR EACH userId IN recipientIds:
    message ← renderTemplate(template, context, userId)
    user ← SELECT email FROM Users WHERE user_id = userId
    // In-app (always)
    InAppNotifier.persist(userId, eventType, message, context.entityId)
    // Email (always, with retry)
    EmailNotifier.send(user.email, template.subject, message)
    // Push (optional, degrade gracefully)
    IF PushNotifier.isAvailable(userId) THEN
      PushNotifier.push(userId, template.subject, message)
    END IF
  END FOR
```

5.5.2 EmailNotifier.send(toEmail, subject, body)

```
FUNCTION send(toEmail, subject, body):
  maxRetries ← 3
  FOR attempt = 1 TO maxRetries:
    TRY
      SMTP.connect(host, port, credentials)
      SMTP.sendMessage(from=SYSTEM_EMAIL, to=toEmail, subject, body)
      LOG delivery success
      RETURN
    CATCH SMTPException AS e
      LOG warning(attempt, e.message)
      WAIT 2^attempt seconds // exponential back-off
    END TRY
  END FOR
  LOG error("Email delivery failed after 3 attempts for " + toEmail)
```

5.5.3 InAppNotifier.persist(userId, eventType, message, entityId)

```
FUNCTION persist(userId, eventType, message, entityId):
  INSERT INTO Notifications (
    recipient_user_id, event_type, message,
    related_entity_id, is_read=FALSE, created_at=NOW())
```

5.6 Admin Dashboard Module

5.6.1 AdminController – authorization guard

```
FUNCTION requireAdminRole(sessionToken):
  userProfile ← SessionManager.validateSession(sessionToken)
```

Software Design Document

```
IF userProfile.role != 'FACULTY_ADMIN' THEN
    RETURN error(403, "Access denied - Faculty Admin role required")
END IF
RETURN userProfile
```

5.6.2 ReportManager.removeReport(reportId, reportType, adminId, reason)

```
FUNCTION removeReport(reportId, reportType, adminId, reason):
    IF reportType == 'LOST' THEN
        report ← SELECT * FROM LostReports WHERE report_id = reportId
        UPDATE LostReports SET is_active=FALSE, status='REMOVED', updated_at=NOW()
            WHERE report_id = reportId
        affectedUserId ← report.owner_user_id
    ELSE
        report ← SELECT * FROM FoundReports WHERE report_id = reportId
        UPDATE FoundReports SET is_active=FALSE, status='REMOVED', updated_at=NOW()
            WHERE report_id = reportId
        affectedUserId ← report.finder_user_id
    END IF
    AuditLogger.log(adminId, 'REPORT REMOVED', reportType, reportId, reason)
    NotificationService.notify('REPORT_REMOVED_BY_ADMIN', [affectedUserId], {reason})
    RETURN success()
```

5.6.3 AnalyticsService.generateSummary(startDate, endDate)

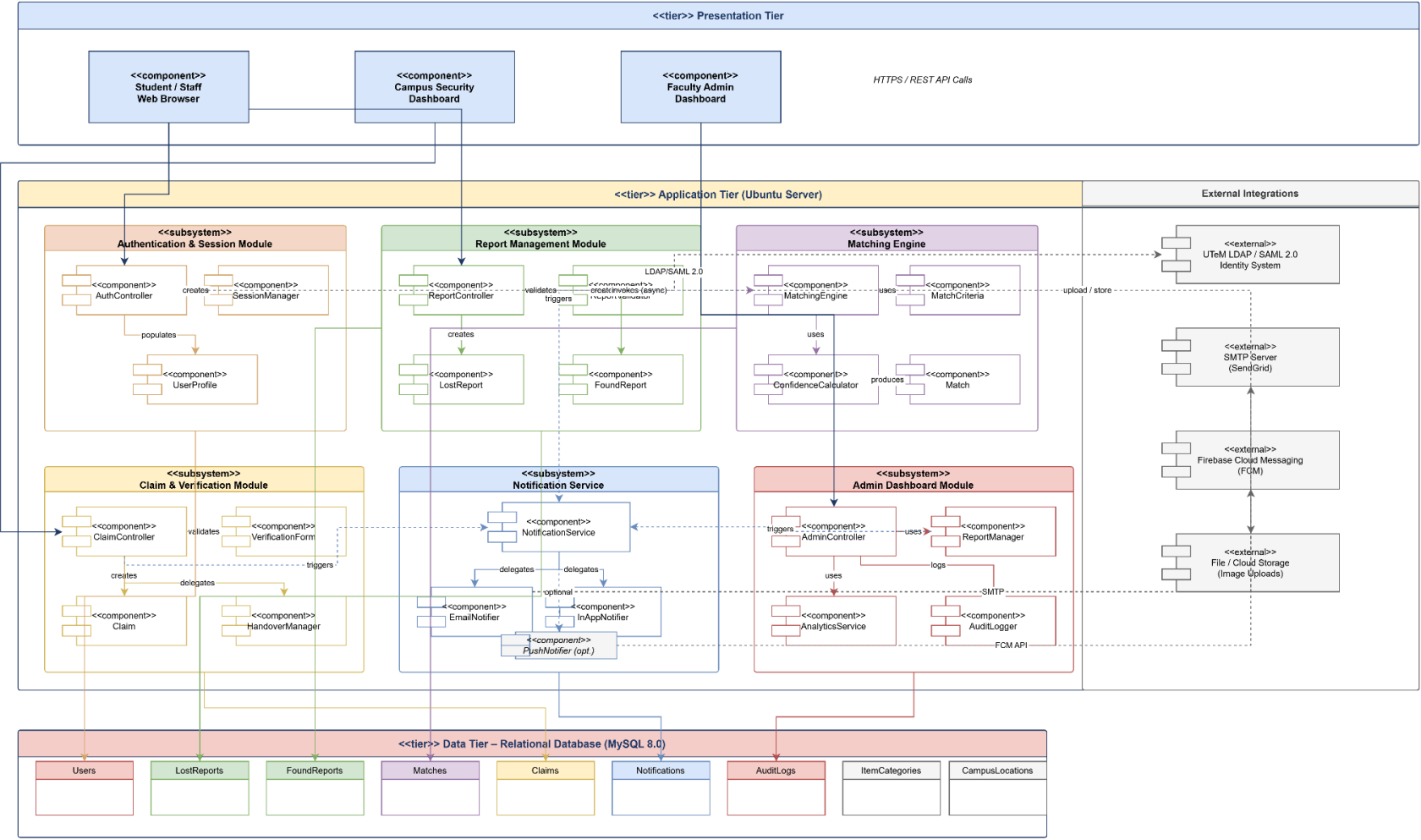
```
FUNCTION generateSummary(startDate, endDate):
    totalLost ← SELECT COUNT(*) FROM LostReports
        WHERE submitted_at BETWEEN startDate AND endDate
    totalFound ← SELECT COUNT(*) FROM FoundReports
        WHERE submitted_at BETWEEN startDate AND endDate
    totalReturned ← SELECT COUNT(*) FROM LostReports
        WHERE status='RETURNED'
        AND closed_at BETWEEN startDate AND endDate
    topLocations ← SELECT cl.location_name, COUNT(*) AS loss_count
        FROM LostReports lr
        JOIN CampusLocations cl ON cl.location_id =
lr.last_known_location_id
        WHERE lr.submitted_at BETWEEN startDate AND endDate
        GROUP BY cl.location_id ORDER BY loss_count DESC LIMIT 5
    topCategories ← SELECT ic.category_name, COUNT(*) AS count
        FROM LostReports lr
        JOIN ItemCategories ic ON ic.category_id = lr.category_id
        WHERE lr.submitted_at BETWEEN startDate AND endDate
        GROUP BY ic.category_id ORDER BY count DESC LIMIT 5
    RETURN SummaryReport{totalLost, totalFound, totalReturned,
        topLocations, topCategories,
        period:{startDate, endDate}}
```

5.6.4 AuditLogger.log(actorId, actionType, entityType, entityId, description)

```
FUNCTION log(actorId, actionType, entityType, entityId, description):
    INSERT INTO AuditLogs (
        actor_user_id, action_type, target_entity_type, target
```

5.8 Component Diagram

CampusFind – Component Diagram



6.0 HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

CampusFind presents three distinct role-based interfaces, each tailored to the needs of its intended user. All interfaces share a common navigation bar displaying the CampusFind logo, the user's name and role badge, a notification bell with unread count, and a logout button. The system applies WCAG 2.1 accessibility principles throughout: sufficient colour contrast, keyboard-navigable controls, and screen-reader-compatible labels.

Student / Staff Interface

Upon login, the user lands on a personalized Dashboard showing:

(1) a summary card for active lost reports they have submitted, (2) a summary card for active found reports they have submitted, and (3) a notification feed. From the dashboard, users can:

- Submit a Lost Report via a multi-field form (title, category drop-down, location drop-down, date picker, description text area, and optional photo upload).
- Submit a Found Report via a similar form with a mandatory photo upload field.
- Browse the searchable Item Listings page, which shows all active found reports filterable by category, location, and date range.
- View their My Reports page, showing all reports they have submitted and their current statuses with status-change timeline.
- View Potential Matches, where the system has identified a likely pairing and displays the confidence score alongside a 'Submit Claim' button.
- Complete an ownership Claim Form, filling in verification fields (brand, serial number, distinguishing features, contents) before submission.
- Track Claim Status : 'Pending Review', 'Approved', or 'Rejected' with decision notes from the finder displayed alongside.
- View Notifications, with each notification linking to the relevant report or claim. Clicking a notification marks it as read.

Software Design Document

Campus Security Interface

Campus Security personnel see a simplified Handover Queue dashboard showing all claims with status 'APPROVED' and awaiting physical handover. For each approved claim, the interface displays: the item description, the claimant's university ID, and the finder's university ID. A 'Confirm Handover' button, protected by a confirmation dialog, triggers `HandoverManager.confirmHandover()` and marks all associated reports as RETURNED.

Faculty Admin Interface

The Faculty Admin Dashboard provides: a statistics overview panel (total lost, found, returned for the current month); a full report management table (all active reports, searchable and filterable); status override controls; a report removal action with mandatory reason entry; and an Analytics page where the admin selects a date range and generates a `SummaryReport` that can be viewed in-browser or downloaded as CSV or PDF.

Across all interfaces, feedback is provided consistently: green toast notifications for successful actions, red inline error messages for validation failures, and yellow banners for warnings (e.g., photo size exceeded). All form submissions are confirmed by a brief loading spinner followed by a success or error toast.

6.2 Screen Images

The wireframes below illustrate the primary screens of the CampusFind interface.

Screen 1 – Register and Login page

CampusFind
UTeM Lost & Found Management System

USER ID

PASSWORD

EMAIL

FACULTY

ROLE

REGISTER

Software Design Document

Figure 6.0 : Register interface

The register interface is a white card centered on a light beige background. At the top is the CampusFind logo. Below it is the system name 'UTeM Lost & Found Management System'. The registration form includes a 'University ID (Matric / Staff ID)' field with the value 'B032420128', a 'Password' field with masked characters, and a 'Sign in via UTeM SSO' button. A privacy notice at the bottom states: 'Protected under PDPA 2010 (Malaysia). Your credentials are authenticated via the UTeM Identity System and are never stored by CampusFind.'

Figure 6.1 : login interface

Screen 2 – Student/Staff/ Citizens Dashboard

The dashboard features a dark blue header with the CampusFind logo, a notification bell with 3 alerts, the user name 'Nurhanim Nabila', and a 'Logout' button. The main content is divided into 'MY REPORTS' and 'NOTIFICATIONS' sections.

MY REPORTS

Count	Report Type	Status	Item	Date
2	My Lost Reports	MISSING	Laptop bag	12 Jun
1	My Found Reports	VERIFIED	Student ID	10 Jun

Below the reports are two buttons: 'Report Lost Item' and 'Report Found Item'.

NOTIFICATIONS

- Potential match found for your **Laptop bag** report — confidence: **HIGH**
2 minutes ago
- A claim was submitted on your found item **Student ID card**
1 hour ago
- Your report **Blue water bottle** has been closed
Yesterday

Figure 6.2 : Student/Staff/ Citizens Dashboard interface

Screen 3 – Submit Lost Report Form

The screenshot shows the 'Report a Lost Item' form within the CampusFind application. The header bar is dark blue with the 'CampusFind' logo on the left, the user name 'Nurhanim Nabila' in the center, and a 'Logout' button on the right. The form itself has a white background with a dark blue back arrow and title. It contains several input fields: 'Item Title' with a placeholder 'e.g. Blue laptop bag', 'Category' and 'Last Known Location' as dropdown menus, 'Date Lost' with a calendar icon and '12/06/2026', and 'Time Lost (optional)' with a clock icon and a dash. There is a large text area for 'Description' with a placeholder 'Describe the item in detail — brand, colour, distinguishing marks...'. Below this is a dashed box for 'Photo (optional, max 2 MB)' with a 'Click to upload image' prompt. At the bottom are two buttons: 'Submit Report' and 'Cancel'.

CampusFind Nurhanim Nabila Logout

< **Report a Lost Item**

Item Title
e.g. Blue laptop bag

Category Last Known Location
Select... Select...

Date Lost Time Lost (optional)
12/06/2026 —

Description
Describe the item in detail — brand, colour, distinguishing marks...

Photo (optional, max 2 MB)
Click to upload image

Submit Report Cancel

Figure 6.3 : Submit Lost Report Form interface

Screen 4 – Submit Found Report Form

CampusFind Nurhanim Nabila [Logout](#)

< Report a Found Item

1 A clear photo helps the owner identify their item and is required for found reports.

Photo of Found Item *

[📷](#) Photo is required — click to upload

📷 A photo of the found item is required.

Item Title

e.g. Samsung Galaxy S23

Category Found Location

Electronics FTMK Library

Date Found Time Found (optional)

📅 12/06/2026 ⌚ 2:15 PM

Storage Location *

e.g. Security Counter Block A

Description

Black phone with cracked screen protector, found near Study Area 3...

Submit Report **Cancel**

Figure 6.4 : Submit Found Report Form interface

Software Design Document

Screen 5 – Item Listings Page

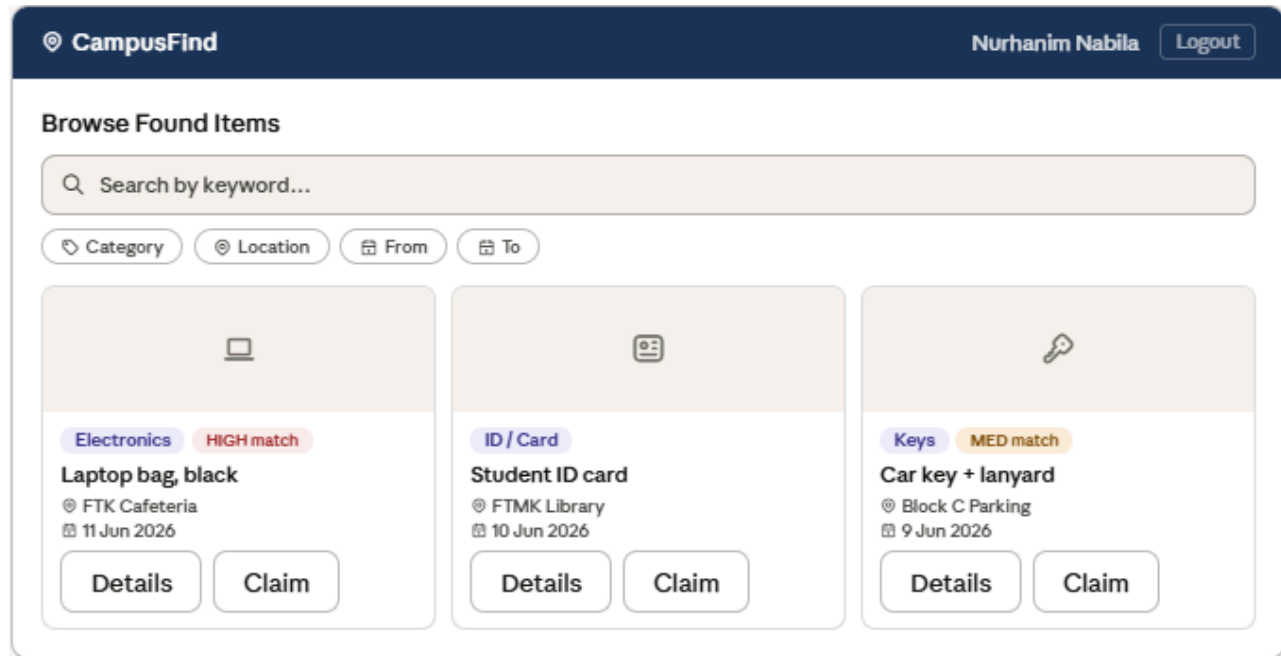


Figure 6.5 : Item Listings Page interface

Software Design Document

Screen 6 – Claim Submission Form

The screenshot shows the 'Submit Ownership Claim' form in the CampusFind app. The header bar is dark blue with the 'CampusFind' logo on the left and the user's name 'Nurhanim Nabila' and a 'Logout' button on the right. The form title is '< Submit Ownership Claim'. The first section, 'FOUND ITEM', contains the text 'Laptop bag, black' and a tag 'Electronics' with a location and date 'FTK Cafeteria · 11 Jun 2026'. Below this is a warning message: 'All fields are required. Submitting false or misleading information is a violation of university policy and may result in disciplinary action.' The form then has several input fields, each with a label and a red asterisk indicating it is required: 'Brand / Model' (filled with 'Targus 15.6" Laptop Backpack'), 'Colour' (filled with 'Black with grey zippers'), 'Serial Number or Unique Identifier' (filled with 'No serial — sticker "FTMK B03" on bottom'), 'Distinguishing Marks' (filled with 'Torn left strap, red keychain attached'), and 'Contents or Secondary Items' (filled with 'MacBook Air M2 (silver), power bank, 3 USB cables, lecture notes folder'). At the bottom of the form is a large white button labeled 'Submit Claim'.

CampusFind Nurhanim Nabila Logout

< **Submit Ownership Claim**

FOUND ITEM
Laptop bag, black
Electronics FTK Cafeteria · 11 Jun 2026

⚠ All fields are required. Submitting false or misleading information is a violation of university policy and may result in disciplinary action.

Brand / Model *
Targus 15.6" Laptop Backpack

Colour *
Black with grey zippers

Serial Number or Unique Identifier *
No serial — sticker "FTMK B03" on bottom

Distinguishing Marks *
Torn left strap, red keychain attached

Contents or Secondary Items *
MacBook Air M2 (silver), power bank, 3 USB cables, lecture notes folder

Submit Claim

Figure 6.6 : Claim Submission Form interface

Software Design Document

Screen 7 – My Reports Page

CampusFind

3

Nurhanim Nabila

Logout

Lost Reports

Found Reports

My Claims

Title	Category	Date	Status	Actions
Laptop bag	Electronics	12 Jun	MISSING	View Matches
Blue water bottle	Other	5 Jun	LOCATED	View Matches
Car key + lanyard	Keys	1 Jun	RETURNED	View Claim

Figure 6.7 : Reports interface

Screen 8 – Faculty Admin Dashboard

CampusFind		Admin		Admin		Logout	
Dashboard		Analytics		Audit Log			
47		39		28		6	
Total Lost (month)		Total Found (month)		Total Returned		Pending Claims	
ID	Type	Title	Category	Status	Actions		
#104	Lost	Laptop bag	Electronics	MISSING	Edit		F
#103	Found	Student ID card	ID / Card	VERIFIED	Edit		F
#102	Lost	Car key + lanyard	Keys	RETURNED	Edit		F

Figure 6.8 : Faculty Admin Dashboard interface

Software Design Document

Screen 9 – Analytics / Summary Report

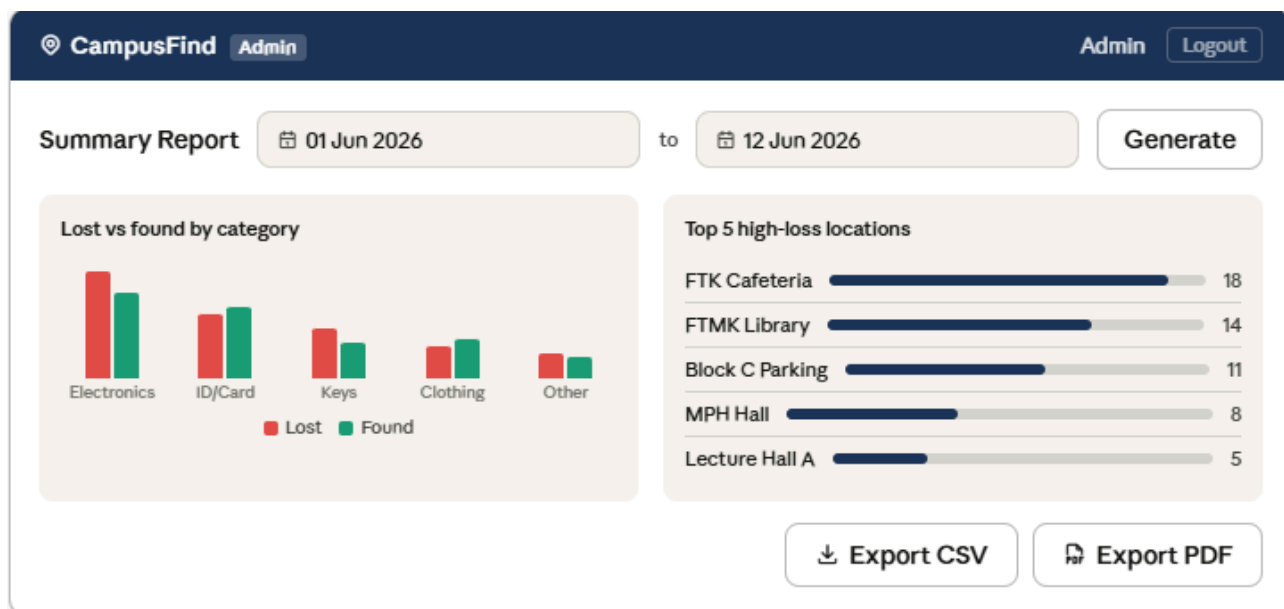


Figure 6.9 :Analytics / Summary Report interface

Screen 10 – Campus Security Handover Queue

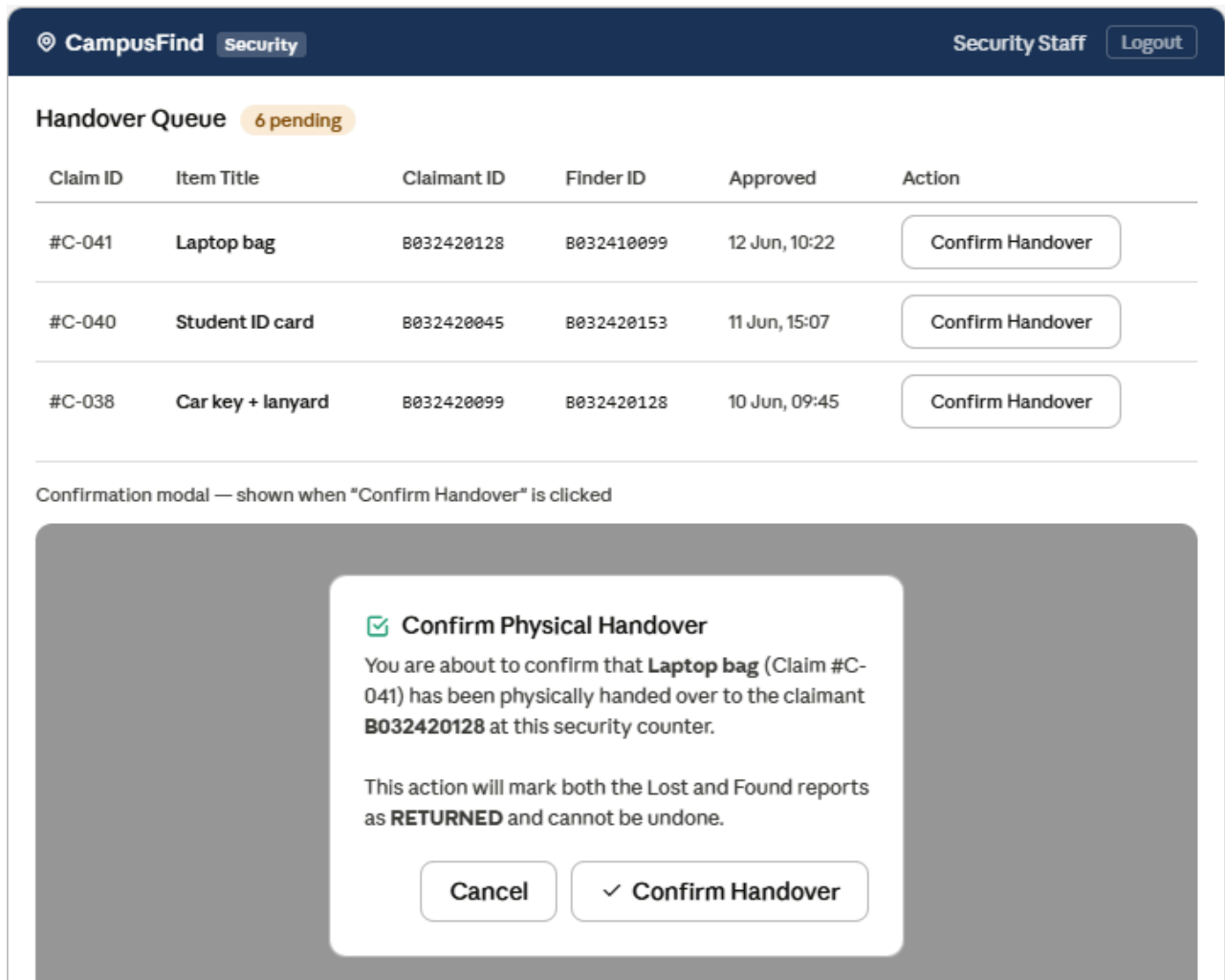


Figure 6.10 : Campus Security Handover Queue interface

6.3 Screen Objects and Actions

The table below lists the primary screen objects across all interfaces and the actions they trigger within the system.

Screen Object	Location	User Action	System Response
Sign In button	Login Page	Click after entering university ID + password	AuthController.login() called; session token created; redirect to Dashboard
Submit button	Register Page	Click after entering all details	AuthController.login() called; session token created; redirect to Dashboard
Report Lost Item button	Dashboard	Click	Navigate to Lost Report form (Screen 3)
Report Found Item button	Dashboard	Click	Navigate to Found Report form (Screen 4)
Category drop-down	Report Forms	Select value	Filters available locations; updates form validation state
Photo upload field	Found Report Form	Select file	Validate file type and size (≤ 2 MB); preview thumbnail displayed
Submit button (Lost Report)	Lost Report Form	Click	ReportValidator runs; on success, record persisted; MatchingEngine invoked async; success toast shown
Submit button (Found Report)	Found Report Form	Click	Photo mandatory check; on success, record persisted; MatchingEngine invoked async; success toast shown
Notification Bell	All screens (nav bar)	Click	Notification panel opens; unread count cleared; each item links to relevant entity
Submit Claim button	Item Listings / My Reports	Click	Navigate to Claim Submission Form (Screen 6) with found report context pre-loaded
Submit Claim form button	Claim Form	Click	VerificationForm.validate(); ClaimController.submitClaim(); finder notified; status badge updated

Software Design Document

Approve Claim button	Finder's My Reports	Click	Confirmation dialog; ClaimController.approveClaim(); claimant and finder notified; handover instructions shown
Reject Claim button	Finder's My Reports	Click	Rejection notes dialog; ClaimController.rejectClaim(); claimant notified
Confirm Handover button	Security Dashboard	Click	Confirmation modal; HandoverManager.confirmHandover(); reports closed; RETURNED notification sent
Remove Report (Admin)	Admin Report Table	Click	Reason entry dialog; ReportManager.removeReport(); audit log entry created; user notified
Edit Status (Admin)	Admin Report Table	Select new status	Status updated in DB; AuditLogger.log() called; user notified if relevant
Generate Report button	Analytics Page	Click after selecting date range	AnalyticsService.generateSummary(); chart and table rendered; CSV/PDF export enabled
Logout button	All screens (nav bar)	Click	SessionManager invalidates token; redirect to Login Page

7.0 REQUIREMENTS MATRIX

This section provides a complete cross-reference tracing every functional and non-functional requirement from the CampusFind Software Requirements Specification (SRS v1.0) to the system component(s), data structure(s), and SDD section(s) that satisfy it. Requirement identifiers follow the codes defined in SRS Section 4 (REQ-x.x) and Section 5 (SRS 5.x).

7.1 Functional Requirements Traceability

Each row maps one SRS functional requirement code to the primary system component(s) responsible for implementing it, the key database table(s) involved, and the relevant SDD section for reference. Requirements are grouped by SRS system feature.

REQ ID	SRS Functional Requirement	Responsible Component(s)	Key Data Table(s)	SDD
Feature 4.1 – User Authentication (Login via University Credentials)				
REQ-1.1	Authenticate users via UTeM Identity System using matric/staff ID and password	AuthController, UTeM SSO (LDAP/SAML 2.0)	Users	3.2.1, 5.1.1
REQ-1.2	Retrieve user's full name, faculty, and account type on successful login	AuthController, UserProfile	Users	3.2.1, 5.1.1
REQ-1.3	Determine role (Student, Staff, Faculty Admin) from UTeM Identity System	AuthController, UserProfile, SessionManager	Users	3.2.1, 5.1.1
REQ-1.4	Deny access and display error message if credentials are invalid	AuthController	Users	5.1.1
REQ-1.5	Deny access and display deactivation message if account is inactive	AuthController	Users	5.1.1
REQ-1.6	Redirect authenticated user to personalised dashboard on success	AuthController, SessionManager	Users	5.1.1, 6.1
Feature 4.2 – Lost Item Reporting (Report Lost Item)				
REQ-2.1	Allow any authenticated user to submit a lost item report	ReportController	LostReports	3.2.2, 5.2.1

Software Design Document

REQ-2.2	Capture: title, category, description, last known location, date lost, optional time, optional photo	ReportController, LostReport entity	LostReports, ItemCategories, CampusLocations	5.2.1
REQ-2.3	Validate all required fields before saving the report	ReportValidator	LostReports	5.2.3
REQ-2.4	Assign 'MISSING' status to a newly submitted lost item report	ReportController	LostReports	5.2.1
REQ-2.5	Make submitted report visible to all authenticated faculty members via search page	ReportController	LostReports	6.1, 6.3
REQ-2.6	Automatically trigger matching process on submission of a new lost item report	ReportController → MatchingEngine (async)	Matches, LostReports, FoundReports	3.2.3, 5.3.1
REQ-2.7	Allow user to edit or close their own lost item report at any time	ReportController (CRUD)	LostReports	5.2.1
REQ-2.8	Accept photo uploads in image format only, max file size 2 MB	ReportValidator, ImageStorageService	LostReports	5.2.3, 5.2.4
Feature 4.3 – Found Item Reporting (Report Found Item)				
REQ-3.1	Allow any authenticated user to submit a found item report	ReportController	FoundReports	3.2.2, 5.2.2
REQ-3.2	Capture: title, category, description, found location, date, optional time, storage location, mandatory photo	ReportController, FoundReport entity	FoundReports, ItemCategories, CampusLocations	5.2.2
REQ-3.3	Require a photo upload before a found item report can be submitted	ReportValidator, ImageStorageService	FoundReports	5.2.2, 5.2.4
REQ-3.4	Validate all required fields before saving the report	ReportValidator	FoundReports	5.2.3

Software Design Document

REQ-3.5	Assign 'Found – Awaiting Claim' status to a newly submitted found item report	ReportController	FoundReports	5.2.2
REQ-3.6	Make submitted found report visible to all authenticated faculty members via search page	ReportController	FoundReports	6.1, 6.3
REQ-3.7	Automatically trigger matching process on submission of a new found item report	ReportController → MatchingEngine (async)	Matches, LostReports, FoundReports	3.2.3, 5.3.1
REQ-3.8	Allow user to update the storage location field of their own found item report	ReportController (CRUD)	FoundReports	5.2.2
REQ-3.9	Accept photo uploads in image format only, max file size 2 MB	ReportValidator, ImageStorageService	FoundReports	5.2.3, 5.2.4
Feature 4.4 – Search and Browse Reports				
REQ-4.1	Allow all authenticated users to search all active lost and found item reports	ReportController	LostReports, FoundReports	6.1, 6.3
REQ-4.2	Support keyword matching against item title and description fields	ReportController (search query)	LostReports, FoundReports	6.1
REQ-4.3	Allow filtering by category, faculty location, date range, and item status	ReportController (filter logic)	LostReports, FoundReports, ItemCategories, CampusLocations	6.1, 6.3
REQ-4.4	Display search results as paginated item cards	Presentation Tier (UI)	LostReports, FoundReports	6.1, 6.2
REQ-4.5	Allow users to click an item card to view full report details	Presentation Tier, ReportController	LostReports, FoundReports	6.1, 6.3
REQ-4.6	Display all active reports newest-first when no keyword or filter is applied	ReportController (default sort)	LostReports, FoundReports	6.1

Software Design Document

REQ-4.7	Display 'No results found' message when no reports match search criteria	Presentation Tier (UI feedback)	—	6.1
REQ-4.8	Return search results within 1 second for up to 1,000 active reports	ReportController, indexed DB columns (category_id, status, submitted_at)	LostReports, FoundReports	3.3 (NFR), 5.1
Feature 4.5 – Automatic Matching and Notification				
REQ-5.1	Automatically trigger matching process whenever a new lost or found item report is submitted	ReportController → MatchingEngine (async invoke)	Matches	5.3.1
REQ-5.2	Identify potential match when reports share same category, same/nearby location, and dates within ± 7 days	MatchingEngine, MatchCriteria	Matches, LostReports, FoundReports, CampusLocations	5.3.1, 5.3.2, 5.3.3
REQ-5.3	Calculate a confidence level for each identified potential match	ConfidenceCalculator	Matches	5.3.4
REQ-5.4	Send email and in-app notification to both users when a potential match is found	NotificationService, EmailNotifier, InAppNotifier	Notifications, Users	3.2.5, 5.5.1
REQ-5.5	Display match and confidence level on both users' dashboards	Presentation Tier, NotificationService	Matches, Notifications	6.1, 6.3
REQ-5.6	Leave report active and re-evaluate when new opposite-type reports are submitted if no match found	MatchingEngine (invoked per new report)	LostReports, FoundReports	5.3.1
REQ-5.7	Automatically retry failed email notifications without user intervention	EmailNotifier (retry with exponential back-off)	Notifications	5.5.2
Feature 4.6 – Submit Claim on Found Item				
REQ-6.1	Allow any authenticated user to submit a claim on a found item report in 'Found – Awaiting Claim' status	ClaimController	Claims, FoundReports	3.2.4, 5.4.1

Software Design Document

REQ-6.2	Require claimant to answer verification questions before claim can be submitted	ClaimController, VerificationForm	Claims	5.4.1
REQ-6.3	Notify finder via email and in-app notification when a claim is submitted	NotificationService, EmailNotifier, InAppNotifier	Notifications	5.5.1
REQ-6.4	Update claim status to 'Pending Review' upon successful submission	ClaimController	Claims	5.4.1
REQ-6.5	Prevent claim submission if the verification form is incomplete	VerificationForm (validation)	Claims	5.4.1
Feature 4.7 – Verify and Approve or Reject Claim				
REQ-7.1	Allow finder to review claimant's verification answers from their dashboard	ClaimController, Presentation Tier	Claims	5.4.2, 6.1
REQ-7.2	Allow finder to approve or reject a submitted claim	ClaimController	Claims, FoundReports	5.4.2
REQ-7.3	Notify both users via email and in-app notification when a claim is approved	NotificationService, EmailNotifier, InAppNotifier	Notifications	5.5.1
REQ-7.4	Display agreed handover location to both users upon claim approval	ClaimController, Presentation Tier	Claims	5.4.2, 6.1
REQ-7.5	Notify claimant via in-app notification when their claim is rejected	NotificationService, InAppNotifier	Notifications	5.5.1
REQ-7.6	Allow finder to mark the item as 'Handed Over' after physical exchange	HandoverManager	Claims, FoundReports	5.4.3
REQ-7.7	Automatically update status to 'RETURNED' and close both reports when item is marked 'Handed Over'	HandoverManager	LostReports, FoundReports, Matches, Claims	5.4.3
REQ-7.8	Send reminder notification to finder if claim has not been responded to within defined period	NotificationService (scheduled reminder)	Notifications, Claims	5.5.1

Software Design Document

REQ-7.9	Maintain item status as 'Found – Awaiting Claim' if claim is rejected	ClaimController	FoundReports, Claims	5.4.2
Feature 4.8 – Track Item Status				
REQ-8.1	Allow authenticated users to view all reports they have submitted or are involved in	ReportController	LostReports, FoundReports, Claims	6.1, 6.3
REQ-8.2	Display current status label for each report (MISSING, Found – Awaiting Claim, Matched, Handed Over, RETURNED)	Presentation Tier	LostReports, FoundReports	6.1, 6.2
REQ-8.3	Display a timeline of all status updates for each selected report	Presentation Tier, ReportController	LostReports, FoundReports, AuditLogs	6.1
REQ-8.4	Display matched report and confidence level when status is 'Matched'; prompt user to submit claim	Presentation Tier, MatchingEngine	Matches	6.1, 6.3
REQ-8.5	Allow user to edit their own report from the status view	ReportController (CRUD)	LostReports, FoundReports	5.2.1, 5.2.2
REQ-8.6	Allow user to close their own report (removes from active search results)	ReportController	LostReports, FoundReports	5.2.1, 5.2.2
REQ-8.7	Display message and shortcuts to report forms when user has no reports	Presentation Tier (UI)	—	6.1
Feature 4.9 – Admin Report Management (Admin Dashboard)				
REQ-9.1	Provide dedicated Admin Dashboard accessible only to Faculty Admin role	AdminController (role guard)	Users	3.2.6, 5.6.1
REQ-9.2	Display all reports with current statuses on Admin Dashboard (active, matched, closed)	AdminController, ReportManager	LostReports, FoundReports	5.6.1, 6.1
REQ-9.3	Allow Faculty Admin to manually update the status of any report	ReportManager, AuditLogger	LostReports, FoundReports, AuditLogs	5.6.2

Software Design Document

REQ-9.4	Allow Faculty Admin to remove invalid or duplicate reports	ReportManager, AuditLogger	LostReports, FoundReports, AuditLogs	5.6.2
REQ-9.5	Notify affected users via in-app notification when their report is removed by Admin	NotificationService, InAppNotifier	Notifications	5.5.1, 5.6.2
REQ-9.6	Allow Faculty Admin to generate a summary report for a specified date range	AnalyticsService, SummaryReport	LostReports, FoundReports, Matches, CampusLocations	5.6.3
REQ-9.7	Compile and display: total lost, total found, total returned, high-loss locations	AnalyticsService	LostReports, FoundReports, Matches, CampusLocations, ItemCategories	5.6.3
REQ-9.8	Make generated summary report available for download (CSV / PDF)	AnalyticsService, SummaryReport	—	5.6.3
REQ-9.9	Display appropriate error message if Admin attempts to access a report that no longer exists	AdminController, Presentation Tier	—	5.6.1

Software Design Document

7.2 Non-Functional Requirements Traceability

The table below traces every non-functional requirement stated in SRS Section 5.0 (Performance, Safety, Security, Software Quality Attributes, Business Rules) to the design decision or system component that satisfies it.

SRS Source	Non-Functional Requirement	Design Decision / SDD Component	SDD
SRS 5.1 – Performance Requirements			
SRS 5.1	Support up to 1,000 active reports without performance degradation	Three-tier architecture; indexed DB columns	3.1, 3.3
SRS 5.1	Search results returned within 1 second ($\leq 1,000$ reports)	MatchCriteria pre-filter; indexed category_id, status, date columns	5.3.1, REQ-4.8
SRS 5.1	Authentication completed within 3 seconds	SSO integration via LDAP/SAML 2.0; AuthController	5.1.1
SRS 5.1	Matching process triggered immediately and completed within 5 seconds	Asynchronous MatchingEngine invocation after report save	3.3, 5.3.1
SRS 5.1	Support at least 300 concurrent users during peak hours	Application Tier scalable independently of Presentation Tier	3.1, 3.3
SRS 5.1	Photo uploads (≤ 2 MB) complete within 5 seconds	ImageStorageService with size validation before upload	5.2.4
SRS 5.1	99% system availability during university operating hours	Deployment-level monitoring; daily automated backups	4.1 (SRS 6.0)
SRS 5.2 – Safety Requirements			
SRS 5.2	Only authenticated users may access the system	SessionManager.validateSession() called on every request	5.1.2, 5.1.3
SRS 5.2	Only report owners or Faculty Admins may edit, close, or remove reports	ReportController ownership check; AdminController role guard	5.2.1, 5.6.1
SRS 5.2	File uploads validated; only supported image formats accepted	ReportValidator, ImageStorageService	5.2.3, 5.2.4
SRS 5.2	Confirmation prompts before any report deletion or admin removal	Presentation Tier confirmation dialog before RemoveReport action	6.3

Software Design Document

SRS 5.2	Backup copies of report data maintained for server recovery	Daily automated backup jobs (deployment layer)	4.1 (SRS 6.0)
SRS 5.2	All administrative actions logged for auditing	AuditLogger (status changes, removals, claim interventions)	5.6.4
SRS 5.2	Compliance with PDPA 2010 (Malaysia)	PII encrypted at rest; AuditLogs maintained; SSO-only auth	3.3, 4.1
SRS 5.3 – Security Requirements			
SRS 5.3	Secure authentication via UTeM Identity System using HTTPS/TLS	AuthController + enforced HTTPS at deployment level	3.3, 5.1.1
SRS 5.3	Role-Based Access Control (RBAC) enforced for all functions	AdminController role guard; session role checked per request	3.2.1, 5.6.1
SRS 5.3	Sessions expire after 15 minutes of inactivity	SessionManager sliding-window expiry (15 min)	5.1.3
SRS 5.3	Sensitive data (matric IDs, emails) securely stored in DB	PII columns encrypted at rest in Users table	4.1
SRS 5.3	Input validation and sanitisation (SQL injection, XSS protection)	ReportValidator; parameterised queries via ORM	5.2.3
SRS 5.3	Activity logs maintained for login attempts and admin actions	AuditLogger, AuditLogs table	5.6.4
SRS 5.4 – Software Quality Attributes			
SRS 5.4	Usability: simple, consistent UI for students, staff, and admins	Presentation Tier (WCAG 2.1 AA, responsive design)	6.1, 6.2, 6.3
SRS 5.4	Reliability: accurate status updates; recovery from minor interruptions	Transactional DB updates; HandoverManager atomic status closure	5.4.3
SRS 5.4	Maintainability: modular architecture, well-documented subsystems	Six discrete subsystems with defined responsibility boundaries	3.3
SRS 5.4	Portability: functions on Chrome, Firefox, Edge, Safari; responsive design	Presentation Tier (standard HTML5/CSS3/JS; no plugins)	6.1
SRS 5.5 – Business Rules			

Software Design Document

SRS 5.5	Only authenticated UTeM students, staff, and Faculty Admins may access system	AuthController, SessionManager	5.1
SRS 5.5	Found item report must include at least one uploaded image	ReportValidator (mandatory photo check)	5.2.2, 5.2.3
SRS 5.5	Users may only edit/close reports they personally created	ReportController ownership check	5.2.1, 5.2.2
SRS 5.5	Faculty Admins may remove duplicate, invalid, or inappropriate reports	AdminController, ReportManager	5.6.2
SRS 5.5	All item claims must undergo verification before approval	ClaimController, VerificationForm	5.4.1, 5.4.2
SRS 5.5	Reports automatically removed from active listings when item is RETURNED	HandoverManager (is_active=FALSE, status='RETURNED')	5.4.3
SRS 5.5	Lost and found reports remain active for max 90 days unless manually closed	Scheduled job (deployment layer); ReportController	6.0 (SRS)

8.0 APPENDICES

Appendix A MySQL Database Schema (DDL)

The following DDL statements are provided as a reference for the development team when initialising the CampusFind database. All tables use InnoDB and utf8mb4 charset.

```
CREATE DATABASE IF NOT EXISTS campusfind_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE campusfind_db;

CREATE TABLE ItemCategories (
  category_id INT PRIMARY KEY AUTO_INCREMENT,
  category_name VARCHAR(100) NOT NULL UNIQUE,
  description TEXT,
  is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE CampusLocations (
  location_id INT PRIMARY KEY AUTO_INCREMENT,
  location_name VARCHAR(150) NOT NULL,
  building_code VARCHAR(20),
  description TEXT,
  is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE Users (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  university_id VARCHAR(20) NOT NULL UNIQUE,
  full_name VARCHAR(200) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  faculty VARCHAR(150),
  role ENUM('STUDENT', 'STAFF', 'CAMPUS_SECURITY', 'FACULTY_ADMIN')
  NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  last_login_at DATETIME,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
  CURRENT_TIMESTAMP
);

CREATE TABLE LostReports (
  report_id INT PRIMARY KEY AUTO_INCREMENT,
  owner_user_id INT NOT NULL,
  title VARCHAR(200) NOT NULL,
  category_id INT NOT NULL,
  description TEXT NOT NULL,
  last_known_location_id INT NOT NULL,
  date_lost DATE NOT NULL,
  time_lost TIME,
  photo_url VARCHAR(500),
  status ENUM('MISSING', 'LOCATED', 'VERIFIED', 'RETURNED', 'REMOVED') DEFAULT
  'MISSING',
  is_active BOOLEAN DEFAULT TRUE,
```

Software Design Document

```
submitted_at DATETIME DEFAULT CURRENT_TIMESTAMP,
updated_at   DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
closed_at    DATETIME,
FOREIGN KEY (owner_user_id) REFERENCES Users(user_id),
FOREIGN KEY (category_id) REFERENCES ItemCategories(category_id),
FOREIGN KEY (last_known_location_id) REFERENCES
CampusLocations(location_id)
);

CREATE TABLE FoundReports (
  report_id          INT PRIMARY KEY AUTO_INCREMENT,
  finder_user_id     INT NOT NULL,
  title              VARCHAR(200) NOT NULL,
  category_id        INT NOT NULL,
  description         TEXT NOT NULL,
  found_location_id  INT NOT NULL,
  date_found         DATE NOT NULL,
  time_found         TIME,
  storage_location    VARCHAR(255),
  photo_url          VARCHAR(500) NOT NULL,
  status             ENUM('LOCATED','VERIFIED','RETURNED','REMOVED') DEFAULT
'LOCATED',
  is_active          BOOLEAN DEFAULT TRUE,
  submitted_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at         DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  closed_at         DATETIME,
  FOREIGN KEY (finder_user_id) REFERENCES Users(user_id),
  FOREIGN KEY (category_id) REFERENCES ItemCategories(category_id),
  FOREIGN KEY (found_location_id) REFERENCES CampusLocations(location_id)
);

CREATE TABLE Matches (
  match_id           INT PRIMARY KEY AUTO_INCREMENT,
  lost_report_id     INT NOT NULL,
  found_report_id    INT NOT NULL,
  confidence_level    ENUM('HIGH','MEDIUM','LOW') NOT NULL,
  match_criteria      JSON,
  status             ENUM('PENDING','RESOLVED','DISMISSED') DEFAULT
'PENDING',
  created_at         DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at         DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  FOREIGN KEY (lost_report_id) REFERENCES LostReports(report_id),
  FOREIGN KEY (found_report_id) REFERENCES FoundReports(report_id)
);

CREATE TABLE Claims (
  claim_id           INT PRIMARY KEY AUTO_INCREMENT,
  found_report_id     INT NOT NULL,
  claimant_user_id    INT NOT NULL,
  verification_answers JSON,
  status             ENUM('PENDING_REVIEW','APPROVED','REJECTED') DEFAULT
'PENDING_REVIEW',
```

Software Design Document

```
finder_decision_notes TEXT,  
submitted_at          DATETIME DEFAULT CURRENT_TIMESTAMP,  
reviewed_at           DATETIME,  
FOREIGN KEY (found_report_id) REFERENCES FoundReports(report_id),  
FOREIGN KEY (claimant_user_id) REFERENCES Users(user_id)  
);  
  
CREATE TABLE Notifications (  
  notification_id      INT PRIMARY KEY AUTO_INCREMENT,  
  recipient_user_id   INT NOT NULL,  
  event_type           VARCHAR(100) NOT NULL,  
  message              TEXT NOT NULL,  
  related_entity_type  VARCHAR(50),  
  related_entity_id    INT,  
  is_read              BOOLEAN DEFAULT FALSE,  
  created_at           DATETIME DEFAULT CURRENT_TIMESTAMP,  
  read_at             DATETIME,  
  FOREIGN KEY (recipient_user_id) REFERENCES Users(user_id)  
);  
  
CREATE TABLE AuditLogs (  
  log_id              INT PRIMARY KEY AUTO_INCREMENT,  
  actor_user_id       INT NOT NULL,  
  action_type         VARCHAR(100) NOT NULL,  
  target_entity_type  VARCHAR(50),  
  target_entity_id    INT,  
  description          TEXT,  
  ip_address          VARCHAR(45),  
  created_at          DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (actor_user_id) REFERENCES Users(user_id)  
);
```

Appendix B Confidence Score Calculation Guide

The following table summarises how the ConfidenceCalculator assigns points and translates the total into a confidence label.

Criterion	Points	Notes
Category match (exact)	+40	Pre-filtered before scoring; always contributes
Exact location match (same location_id)	+30	–
Adjacent location match (same building_code)	+15	Applied when exact location does not match
Date difference ≤ 1 day	+20	–
Date difference 2–3 days	+10	–
Date difference 4–7 days	+0	Criteria still met, but no extra score
Description keyword overlap ≥ 3 words	+10	Stopwords excluded from comparison
TOTAL $\geq 80 \rightarrow$ HIGH confidence		Match presented prominently; users strongly encouraged to follow up
TOTAL 55–79 $\rightarrow$ MEDIUM confidence		Match listed; users encouraged to review details
TOTAL $< 55 \rightarrow$ LOW confidence		Match listed but labelled as tentative